FreeCAD Python CUIコマンドによるパーツの色変更の実現可能性に関する専門家レポート

1. はじめに: FreeCADにおけるPythonスクリプトと外観管理の概要

FreeCADは、その統合されたPythonインタープリタを通じて高度にスクリプト化可能なプラットフォームとして設計されています。この設計により、ユーザーはグラフィカルユーザーインターフェース (GUI) を直接操作することなく、Pythonコードを用いてFreeCADドキュメント内の様々な要素をプログラム的に作成、変更、および分析することが可能です。このプログラムによる制御は、単純なジオメトリの作成にとどまらず、Pythonの汎用性を活用した複雑な操作にまで及びます。FreeCADの内部ツール、モジュール、さらにはワークベンチの大部分がPythonで実装されており、FreeCADエコシステムにおけるPythonスクリプトの深い統合と強力な能力を明確に示しています [1]。

本レポートにおける主要な問いは、「FreeCAD Python CUIコマンドでパーツの色を変えることが可能か？」というものであり、これはコマンドラインユーザーインターフェース (CUI) またはヘッドレス環境でFreeCADを実行しながら、視覚的プロパティ、特にパーツの色を変更する可能性に焦点を当てています [1]。この問いに対する初期の回答としては、FreeCAD Pythonコマンドを使用してパーツの色を変更することは可能であるものの、その方法論と変更の即時的な視覚的影響は、FreeCADが完全なGUIで動作しているか、ヘッドレス (CUI) 環境で動作しているかによって大きく異なることが挙げられます。FreeCADが視覚的プロパティをどのように処理し、永続化するかに根本的な違いがあるため、本レポートではこれらの区別を詳細に説明し、両方のシナリオに対する実用的な解決策を提供します [1]。

FreeCADの設計思想は、オブジェクトのコアな幾何学的およびデータ定義と、画面上でのその視覚的表現との厳密な分離に基づいています。このアーキテクチャ上の分離は、主にAppとGuiという2つのPythonモジュールによって管理されています [1]。このAppとGuiの分離は、FreeCADのスクリプト作成、特に完全なGUIなしの環境でモデルを操作したり、モデルに影響を与えたりする場合に極めて重要です [1]。

ユーザーの質問の核心は「スクリプトの埋め込み」という表現にありますが、FreeCADのセキュリティモデルとコード・データ分離の原則から、スクリプト自体をFCStdファイルに直接埋め

込むことはできません[1]。FCStdファイルは、悪意のあるコードの実行を防ぐために、実行可能なPythonコードを直接含みません。したがって、ユーザーが意図する「色をつける」という操作は、コードの埋め込みではなく、色情報の「データ」としての永続化と、そのデータを解釈する外部スクリプトの連携によって実現される必要があります。この基本的な違いを理解することが、FreeCADにおける色変更のスクリプト化の全体像を把握する上で不可欠です。

2. FreeCADのコアアーキテクチャ: データと視覚表現の厳密な分離

FreeCADの堅牢な設計は、オブジェクトのデータ表現と視覚表現を厳密に分離している点にあります。この分離は、PythonのAppモジュールとGuiモジュールによって管理されており、スクリプトによる操作の挙動に根本的な影響を与えます。

Appモジュールと「データプロパティ」の役割（永続性とCUIアクセス）

Appモジュール（例: App.Document、App.DocumentObject、Part::Feature）は、オブジェクトの「何であるか」を定義する役割を担います。これには、そのジオメトリ、寸法、パラメトリックな関係、およびその他の基盤となるデータが含まれます[1]。Appモジュールによって管理されるプロパティは「データプロパティ」と呼ばれ、これらはオブジェクトの存在に不可欠な要素です。データプロパティは、FreeCADがコンソールモード (CUI) またはGUIなしのライブラリとして実行されている場合でも完全にアクセス可能であり、モデルの永続的で非視覚的な側面を表します。これらのプロパティは、ファイル保存時にFCStdドキュメントに永続化されます[1]。具体的な例としては、

Length、Width、Height、Shape、そしてApp.Materialプロパティ（DiffuseColor、AmbientColorなど）が挙げられます[1]。

Guiモジュールと「ビュープロパティ」の役割（GUI依存性）

対照的に、Guiモジュール（例: Gui.Document、Gui.ViewObject、ViewProvider）は、3Dビューでのオブジェクトの視覚的「どのように見えるか」を定義します[1]。これには、色（

ShapeColor）、線幅、表示モード（例:ワイヤーフレーム、シェーディング）、透明度などのプロパティが含まれます。これらは「ビュープロパティ」として知られており、グラフィカルユーザーインターフェースに本質的に結びついています [1]。したがって、Guiオブジェクトおよびそれに関連するViewProviderは、FreeCAD GUIがアクティブにロードされ、実行されている場合にのみ利用可能でアクセス可能です [1]。

CUI/ヘッドレス環境におけるこの分離の含意

このAppとGuiの分離を深く理解することは、FreeCADのスクリプト作成、特に完全なGUIなしの環境でモデルを操作したり、モデルに影響を与えたりする場合に極めて重要です [1]。データプロパティはFreeCADがコンソールモードで使用されている場合やライブラリとして使用されている場合でも常に存在するのに対し、ビュープロパティはグラフィカルユーザーインターフェース (GUI)がロードされている場合にのみアクセス可能であり、FreeCADをコンソールモードやヘッドレスライブラリとして使用している場合はアクセスできません [1]。この事実は、CUIでの色変更の実現可能性に直接影響を与えます [1]。

ユーザーの主な関心事は「CUIコマンド」を使用したパーツの色変更であり、これはヘッドレスまたはコンソール環境を意味します。複数の情報源は、色、線幅などの視覚的プロパティ（「ビュープロパティ」と呼ばれる）およびそれらを管理するオブジェクト（Guiモジュール、ViewProvider）が、GUIがロードされている場合にのみ利用可能であると一貫して明示的に述べています。特に、最も直接的な色プロパティであるShapeColorは、Guiプロパティであることが示されています [1]。ヘッドレス環境では、

Gui.activeDocument()はNoneを返します。また、ViewProviderオブジェクトは作成もアクセスもできません [1]。したがって、純粋なCUI/ヘッドレスFreeCADインスタンスでPythonコマンドを介して

ShapeColorやその他のViewプロパティに直接アクセスまたは変更しようとすると、AttributeErrorなどのエラーが発生し、本質的に失敗します [1]。

このアーキテクチャ上の設計は、コアデータと視覚的表現の意図的な分離を反映しています。この設計は、GUIが不要なバッチ処理、シミュレーション、サーバーサイドでのモデル生成など、視覚的レンダリングを必要としない非視覚的オペレーションには非常に有益です。GUIコンポーネントをロードしないことで、これらのシナリオでのリソース消費を抑え、効率を最大化することが可能になります。しかし、この最適化は、ヘッドレスコンテキストでの直接的な視覚的操作には根本的な制限を課します。このことは、CUIで「色変更」を実現するには異なるアプローチが必要であることを意味します。すなわち、Appレベルで色データを定義し、それが視覚

的環境が利用可能になったときにGuiによって解釈され、レンダリングされるという方法です [1]。

FreeCADにおけるデータプロパティとビュープロパティの主要な区別は、以下の表にまとめられています。この表は、プロパティのアクセス可能性と永続性を支配する基盤となるメカニズムを理解するというユーザーの暗黙のニーズに直接対応し、CUI環境で特定の操作がなぜ可能であるか、あるいは不可能であるかを明確にします [1]。

Table 1: FreeCADオブジェクトプロパティカテゴリ

カテゴリ	モジュール	説明	アクセシビリティ	例
データプロパティ	App	オブジェクトの「物理的」または本質的な特性とジオメトリを定義します(例:寸法、トポロジカル形状、マテリアルプロパティ)。	ヘッドレス/コンソールモードを含む常にアクセス可能。ファイル保存時に永続化されます。	Length, Width, Height, Shape, App.Materialプロパティ(DiffuseColor, AmbientColor)。
ビュープロパティ	Gui	3Dビューでのオブジェクトの視覚的表現を定義します(例:色、線幅、表示モード)。ViewObjectまたはViewProviderに結びついています。	GUIがロードされている場合にのみアクセス可能。ヘッドレス/コンソールモードではアクセス不可。デフォルトでは元に戻すスタックの一部ではありません。	ShapeColor, Line Color, Point Color, Display Mode, Transparency(ビューレベル)。
[1]				

3. GUI環境におけるパーツの色変更の実現

FreeCADのGUI環境がアクティブな場合、パーツの色を変更するためのいくつかの方法が存在します。これらの方法は、即時的な視覚的フィードバックを提供しますが、その永続性やヘッドレス環境での利用可能性には違いがあります。

Shape Colorプロパティの直接操作と視覚的フィードバック

FreeCAD GUIがアクティブな場合、オブジェクトの全体的な色を変更する最も直接的な方法はShapeColorプロパティを使用することです [1]。

ShapeColorはオブジェクトのViewObjectのプロパティであり、Gui.activeDocument().getObject("ObjectName")を介してアクセスされます [1]。色は、赤、緑、青、およびオプションでアルファ(透明度)の浮動小数点値のタプルとして表現され、各コンポーネントは0.0から1.0の範囲です（例:

(R,G,B)または(R, G, B, A)） [1]。

ShapeColorへの変更は通常、FCStdドキュメントに保存されますが [1]、ビュープロパティであるため、そのアクセス可能性と変更はGUIがロードされていることに厳密に結びついています [1]。これは、GUIモードで設定した

ShapeColorがFCStdファイルに保存されたとしても、そのファイルがCUIで開かれた場合には、その色情報が直接利用できないことを意味します。そのため、即時的な視覚フィードバックは得られますが、自動化やヘッドレス処理においては不十分であると評価されます。

App.Materialプロパティの利用と**GUI**での解釈

FreeCADオブジェクトは、Appレベルで定義されたマテリアルプロパティを持つこともでき、これらはその視覚的外観に影響を与えます。これらのプロパティは、永続性とヘッドレス操作に関してより堅牢な方法を提供します [1]。

App.Materialオブジェクトには、DiffuseColor、AmbientColor、SpecularColor、EmissiveColor、SectionColor、Shininess、Transparencyなどのグラフィカルプロパティが含まれます [1]。

これらのプロパティはデータプロパティであり、オブジェクトの基本的な定義の一部であるため、ヘッドレスモードでもアクセスおよび変更が可能です [1]。オブジェクトの

ViewProvider（GUI部分）は、これらのApp.Materialプロパティを解釈して、3Dビューでオブジェクトをレンダリングします [1]。特定の複雑なオブジェクトタイプでは、

App.Materialが外観を制御するためのより信頼性の高い方法である可能性が示唆されています[1]。GUIモードで作業している場合でも、将来的な自動化やCUI互換性を考慮するならば、

ShapeColorよりもApp.Materialを使用する方が永続的な色ソリューションとして優れていると判断されます。

コード例: GUIモードでのShapeColorおよびApp.Materialの設定

以下のコード例は、GUIモードでのShapeColorの直接操作と、App.Materialプロパティの設定方法を示しています。

ShapeColor設定のコード例

Python

```python
import FreeCAD as App
import FreeCADGui as Gui
import Part

# ドキュメントとオブジェクトが存在することを確認
doc = App.newDocument("Color TestDoc")
box = doc.addObject("Part::Box", "MyBox")
box.Shape = Part.makeBox(10, 10, 10)
doc.recompute()

# オブジェクトのGUI表現にアクセス (これはFreeCADがGUIで実行されている場合にのみ機能)
try:
    gui_doc = Gui.activeDocument()
    view_object = gui_doc.getObject("MyBox")
    # 現在の色の出力(例:デフォルトのグレーの場合は (0.8, 0.8,0.8))
    print(f"Current ShapeColor: {view_object.ShapeColor}")
    # シェイプの色を赤に設定 (R=1.0, G=0.0, B=0.0)
```

```
    view_object.ShapeColor = (1.0, 0.0, 0.0)
    print(f"New ShapeColor: {view_object.ShapeColor}")
    # オプション: 透明度の設定(例:50%透明の場合は0.5)
    # view_object.Transparency = 50 # 透明度は通常GUIプロパティでは0-100
    # Shape Colorタプルの場合、アルファは0.0-1.0
    view_object.ShapeColor = (1.0, 0.0, 0.0, 0.5) # 50%透明の赤
    Gui.updateGui() # 3Dビューを更新して変更を反映
    doc.recompute() # ジオメトリの変更や依存関係がある場合はドキュメントを再計算
    print("Shape color changed to red with 50% transparency.")
except AttributeError:
    FreeCAD.Console.PrintError("GUIがアクティブでないか、オブジェクトが見つかりません。ShapeColorはGUIプロパティです。\n")
```

1

App.Materialプロパティ設定のコード例

Python

```
import FreeCAD as App
import FreeCADGui as Gui
import Part

# ドキュメントとオブジェクトが存在することを確認
doc = App.newDocument("Material TestDoc")
box = doc.addObject("Part::Box", "MyMaterialBox")
box.Shape = Part.makeBox(10, 10, 10)
doc.recompute()

# 新しいApp.Materialオブジェクトを作成 (これはAppレベルのデータオブジェクト)
material = App.Material()
# マテリアルプロパティを設定(これらはデータプロパティ)
material.DiffuseColor = (0.0, 0.0, 1.0) # 青
material.Transparency = 0.5 # 50%透明 (0.0から1.0)
```

```
material.AmbientColor = (0.1, 0.1, 0.1) # わずかなアンビエントライト効果

# マテリアルをオブジェクトのViewObjectに割り当てる。
# 割り当て自体はビュープロパティだが、App.Materialデータを参照する。
try:
    view_object = Gui.activeDocument().getObject("MyMaterialBox")
    view_object.Material = material # App.Materialオブジェクトを割り当て
    Gui.updateGui()
    doc.recompute()
    print("MyMaterialBoxのマテリアルプロパティが設定されました。")
except AttributeError:
    FreeCAD.Console.PrintError("GUIがアクティブではありません。ViewObjectにマテリアルを直接割り当てることはできません。\n")
# マテリアルをオブジェクトと共に保存するには、Appオブジェクトのプロパティであるか、
# 保存されるビュープロパティによって参照される必要がある。
# Part::Featureの場合、View.MaterialプロパティはApp.Materialを参照する。
# マテリアルオブジェクト自体はAppレベルの構成要素である。
```

[1]

4. ヘッドレス（CUI）環境における外観管理の戦略と課題

FreeCADをヘッドレス（CUI）モードで操作する場合、GUI環境とは異なるアプローチが必要となります。これは、FreeCADのアーキテクチャがGUIコンポーネントをロードしないことで、リソースを節約し、バッチ処理やサーバーサイドでのモデル生成といった非視覚的オペレーションの効率を最大化しているためです。

「ヘッドレス」FreeCADの定義とGuiモジュールへのアクセス制約

FreeCADを「ヘッドレス」モードで実行するとは、グラフィカルユーザーインターフェースなしで操作することを意味します。これは通常、FreeCADCmd実行可能ファイルを使用するか、-c（コンソール）引数付きでFreeCADを起動することによって行われます[1]。Pythonスクリプトでは、これは完全なGUI環境を初期化しないことと同義です。

import FreeCADGui as Guiは可能ですが、重要なGUIコンポーネントは利用できません [1]。

ビュープロパティとViewProviderオブジェクト（視覚的表現を管理するもの）は、アクティブなGUIに本質的に結びついています。FreeCADがヘッドレスで実行される場合、GUIドキュメントは作成されず、結果としてAppドキュメント内のフィーチャに対応するViewProviderオブジェクトは存在しません [1]。

Gui.activeDocument()にアクセスしようとするとNoneが返されます [1]。

Gui.setupWithoutGUI()を使用すると、FreeCADGuiをロードしながらも、GUIドキュメントやビュープロバイダーを作成しないことが可能であるという点が特筆されます。これは単にウィンドウが非表示になっているというだけでなく、基本的なGUIインフラストラクチャが存在しないことを意味します [1]。

ShapeColorおよびその他のビュープロパティの直接的な無効性

ShapeColorはビュープロパティであるため、ヘッドレス環境でobj.ViewObject.ShapeColorまたはGui.activeDocument().getObject("ObjectName").ShapeColorを設定しようとすると、エラー（例: ViewObjectまたはGui.activeDocument()がNoneであるためのAttributeError）が発生します [1]。同様に、

Line Color、Point Color、Display Mode、Transparencyなどの他の視覚的プロパティ（ViewObjectに直接設定される場合）もアクセスできません [1]。

コード例:ヘッドレスモードでの**Shape Color**の無効性の実証

以下のコード例は、ヘッドレスモードでShapeColorにアクセスしようとした際にエラーが発生することを示しています。

Python

```
import FreeCAD as App
```

```python
import FreeCADGui as Gui # インポートされていても、アクティブなGUI要素は持たない
import os # 一時ファイルのパス操作のため

# GUIなしで明示的にセットアップすることでヘッドレス環境をシミュレート
try:
    Gui.setupWithoutGUI()
    FreeCAD.Console.PrintMessage("FreeCAD GuiがGUIなしでセットアップされました。\n")
except Exception as e:
    FreeCAD.Console.PrintMessage(f"GUIなしでGUIをセットアップできませんでした: {e}。既にヘッドレスと仮定します。\n")

doc = App.newDocument("HeadlessColorTest")
box = doc.addObject("Part::Box", "MyHeadlessBox")
box.Shape = Part.makeBox(10, 10, 10)
doc.recompute()

# Gui.activeDocument()へのアクセスを試みる - Noneになるはず
gui_doc = Gui.activeDocument()
if gui_doc is None:
    FreeCAD.Console.PrintMessage("Gui.activeDocument()はヘッドレスモードではNoneです(期待通り)。\n")
else:
    FreeCAD.Console.PrintMessage("Gui.activeDocument()は予期せずNoneではありません。GUIがアクティブかもしれません。\n")

# Shape Colorにアクセスして変更を試みる - 失敗するはず
try:
    # この行がエラーを発生させるはず
    view_object = Gui.activeDocument().getObject("MyHeadlessBox")
    view_object.ShapeColor = (1.0, 0.0, 0.0)
    FreeCAD.Console.PrintMessage("Shape Colorの設定を試みました(この行は到達しないはずです)。\n")
except AttributeError as e:
    FreeCAD.Console.PrintMessage(f"エラー: {e}。ヘッドレスモードではViewObjectまたはShapeColorにアクセスできません。\n")
except Exception as e:
    FreeCAD.Console.PrintMessage(f"予期せぬエラーが発生しました: {e}\n")

# ドキュメントを保存(このセッションからのShapeColorの変更は含まれない)
temp_file_path = os.path.join("/tmp", "headless_box.FCStd") # 一時パスを使用
```

```
doc.saveAs(temp_file_path)
FreeCAD.Console.PrintMessage(f"ドキュメントが保存されました。ShapeColorを介した色変更は適用されませんでした。パス: {temp_file_path}\n")
```

[1]

データプロパティとしての外観データの永続化

ヘッドレススクリプトで色を定義し、後でFreeCAD GUIでドキュメントを開いたときにその色が表示されるようにするための推奨されるアプローチは、色情報をデータプロパティとして保存することです [1]。

App.Materialの使用**:** ヘッドレスでの色データ定義の主要な方法

App.Materialオブジェクトを作成し、そのDiffuseColor、AmbientColorなどのプロパティを設定します [1]。

App.Materialオブジェクト自体はAppレベルのデータ構成要素であり、そのプロパティ(DiffuseColorなど)はデータプロパティであるため、ヘッドレスモードでアクセスおよび変更が可能です [1]。オブジェクトの

ViewObjectのMaterialプロパティ(例: obj.ViewObject.Material)はビュープロパティであり、ヘッドレスモードで直接設定することはできませんが、標準のPart::FeatureなどのAppオブジェクトは、App.Materialオブジェクトを参照するMaterialプロパティを持つことがよくあります [1]。このデータプロパティはドキュメントと共に保存され、GUIモードでロードされると、

ViewProviderが割り当てられたApp.Materialのプロパティを自動的に解釈し、適用します [1]。これは、ヘッドレス環境で実行する必要があるFreeCADスクリプトで永続的な色情報を定義するための主要な方法です。これにより、焦点が直接的な視覚的操作からデータ駆動型の外観定義へと移ります [1]。

カスタム**App::PropertyColor**の追加**:** 柔軟な色データ保存

App.Materialに加えて、ユーザーはApp::PropertyColorタイプまたはApp::PropertyFloatタイプ（透明度用）のカスタムデータプロパティを、自身のAppオブジェクトに直接追加できます[1]。これらのカスタムプロパティはドキュメントと共に保存され、ヘッドレスでアクセス可能です[1]。ドキュメントがGUI付きのFreeCADで開かれた場合、カスタムの

ViewProviderまたはマクロがこれらのデータプロパティを読み取り、ビュープロパティ（ShapeColor、Transparency）に適用できます[1]。これには、GUI環境内での追加のスクリプト作成またはカスタム

ViewProviderの定義が必要です[1]。

コード例: ヘッドレスモードでの App.Material およびカスタムプロパティの設定(永続性のため)

以下のコード例は、ヘッドレスモードでApp.Materialおよびカスタムプロパティを使用して色情報を永続化する方法を示しています。

Python

```python
import FreeCAD as App
import Part
import os

# Appレベルの操作にはFreeCADGuiのインポートやsetupWithoutGUI()は不要
# このスクリプトはFreeCADCmdまたは'FreeCAD -c'で実行されるように設計されています
doc = App.newDocument("Headless Material Doc")
box = doc.addObject("Part::Box", "MyPersistentBox")
box.Shape = Part.makeBox(10, 10, 10)

# App.Materialオブジェクトを作成(これはAppレベルのデータオブジェクト)
material = App.Material()
material.DiffuseColor = (0.0, 1.0, 0.0) # 緑
material.Transparency = 0.3 # 30%透明
```

```
# Appオブジェクトにカスタムカラープロパティを追加
# これはデータプロパティであり、永続化され、ヘッドレスでアクセス可能
box.addProperty("App::PropertyColor", "DesiredColor", "Appearance", "表示用の色
").DesiredColor = (0.0, 1.0, 0.0)
box.addProperty("App::PropertyFloat", "DesiredTransparency", "Appearance", "表示用の透明度
").DesiredTransparency = 0.3

doc.recompute()

temp_file_path = os.path.join("/tmp", "headless_material_box.FCStd")
doc.saveAs(temp_file_path)

FreeCAD.Console.PrintMessage(f"ドキュメントがカスタムの色/透明度プロパティと共に保存されました。
パス: {temp_file_path}\n")
# このドキュメントがGUIで開かれた場合、カスタムのViewProviderまたはマクロが
# 'DesiredColor'と'DesiredTransparency'を読み取り、実際の視覚的表現
# (ShapeColor, Transparency)に適用できる。
```

[1]

シーングラフ表現の生成: ヘッドレス環境からの視覚出力

CUIのインタラクティブな意味で直接「色を変更する」わけではありませんが、FreeCADはヘッドレスモードでもオブジェクトのシーングラフ表現を生成できます [1]。

Gui.setupWithoutGUI()の後に利用可能なGui.subgraphFromObject(obj)関数は、Coin3Dシーングラフを作成します [1]。このシーングラフは、Appオブジェクトのプロパティ（設定されている場合は

App.Materialなど）やデフォルトのレンダリングルールから派生した外観データを取り込むことができます [1]。このシーングラフは、完全なFreeCAD GUIを表示することなく、画像レンダリング（例: PivyとCoin3Dの機能を使用）に利用できます [1]。

コード例:ヘッドレスモードでのシーングラフ生成(概念的)

以下のコード例は、ヘッドレスモードでシーングラフを生成する概念を示しています。

Python

```python
import FreeCAD as App
import FreeCADGui as Gui
import Part
from pivy import coin # シーングラフ操作/エクスポートに必要
import os

try:
    Gui.setupWithoutGUI()
    FreeCAD.Console.PrintMessage("シーングラフ生成のためにFreeCAD GuiがGUIなしでセットアップされました。\n")
except Exception as e:
    FreeCAD.Console.PrintMessage(f"GUIなしでGUIをセットアップできませんでした: {e}。既にヘッドレスと仮定します。\n")

doc = App.newDocument("ScenegraphRenderDoc")
box = doc.addObject("Part::Box", "MyRenderBox")
box.Shape = Part.makeBox(10, 10, 10)

# カスタムプロパティを設定。これはAppレベルのデータであり、シーングラフが読み取れる。
box.addProperty("App::PropertyColor", "RenderColor", "Render", "ヘッドレスレンダリング用の色").RenderColor = (1.0, 0.5, 0.0) # オレンジ
doc.recompute()

# シーングラフを生成
# この関数は、オブジェクトの現在の状態(Appデータを含む)とデフォルトのレンダリングルールに基づいてCoin3Dグラフを作成する。
scenegraph = Gui.subgraphFromObject(box)
if scenegraph:
    FreeCAD.Console.PrintMessage("シーングラフが正常に生成されました。\n")
    # 例:シーングラフをファイルに保存(例: Inventor形式の.iv)
    # scenegraph.write("output_box.iv") # このメソッドは直接利用できないか、単純ではない可能性がある。
    # Pivyを使用してシーングラフを走査・操作したり、画像にレンダリングしたりできる。
    # 例(概念的):
```

```
    # from pivy.coin import SoOffscreenRenderer
    # renderer = SoOffscreenRenderer(scenegraph)
    # image = renderer.render()
    # image.save("rendered_box.png") # 画像処理ライブラリが必要
else:
    FreeCAD.Console.PrintMessage("シーングラフの生成に失敗しました。\n")

temp_file_path = os.path.join("/tmp", "scenegraph_box.FCStd")
doc.saveAs(temp_file_path)
```

[1]

ヘッドレスモードにおけるデータ永続性と即時視覚フィードバック

ユーザーの質問は「CUIコマンド」に関するものであり、分析により、ヘッドレスモードでは直接的な視覚的変更（ShapeColor）は不可能であることが示されています [1]。しかし、

App.MaterialやカスタムのApp.PropertyColorは色データを永続的に保存できます。これは、ヘッドレスモードでは色データを「定義」できるものの、その変更を即座に「見る」ことはできないことを意味します [1]。視覚的なフィードバックは、ドキュメントがGUIで開かれた場合にのみ発生します [1]。

この事実は、CUIコンテキストでの「色変更」に対するユーザーの理解を再定義します。ユーザーはCUIでの「色変更」を「リアルタイムの視覚的フィードバック」と捉えている可能性がありますが、FreeCADのアーキテクチャはこれを不可能にしています。CUIでの「色変更」は、インタラクティブな視覚的体験ではなく、プログラム的にモデルのデータ構造に色情報を「埋め込む」行為であり、その視覚化は後続のGUIセッションに委ねられます。このCUIでの色変更の再定義は、ユーザーがFreeCADのヘッドレス機能を効果的に活用するためのメンタルモデルを構築する上で不可欠です。

Gui.subgraphFromObject()関数は、ヘッドレスモードから視覚的表現（シーングラフ）を生成する方法を提供し、それを画像としてレンダリングすることができます [1]。これは、ヘッドレスでのデータ操作と視覚出力の間のギャップを埋めるものです [1]。この能力は、ヘッドレス環境でも視覚的な成果物を生成できることを意味し、自動化されたレンダリングパイプラインにとって重要です。

FreeCADのPython APIで期待される色値の正確なデータ型と範囲に関する迅速で明確な参

照は、以下の表にまとめられています。これは、開発者が試行錯誤することなく色変更を正しく実装し、スクリプトが構文的および意味的に正しいことを保証するために不可欠です [1]。

Table 2:一般的な色プロパティ形式

プロパティタイプ	形式	範囲	例
ShapeColor (ビュープロパティ)	(R, G, B) または (R, G, B, A)の浮動小数点タプル	各コンポーネントで0.0から1.0	赤の場合 (1.0, 0.0,0.0)、半透明の緑の場合 (0.0, 1.0, 0.0, 0.5)
App. Materialプロパティ (DiffuseColorなど) (データプロパティ)	(R, G, B)の浮動小数点タプル	各コンポーネントで0.0から1.0	青の場合 (0.0, 0.0, 1.0)
Transparency (ViewObject上のビュープロパティ)	浮動小数点(一部のコンテキストでは整数だが、通常はViewObject.Transparencyで0.0-1.0または0-100)	0.0 (不透明)から1.0 (完全に透明) または0から100	50%透明の場合 0.5
[1]			

5. FCStdファイルへのPythonスクリプトの埋め込みに関する考察

ユーザーの質問には「スクリプトを事前にFCstdに埋め込んでおく方法はあるか？」という要素が含まれていましたが、FreeCADの設計思想とセキュリティモデルは、この直接的な埋め込みを許可していません。

FreeCADのセキュリティモデルとコードとデータの分離

FreeCADのFCStdファイルは「セキュリティ上の理由から、埋め込みコードを一切含まない。パ

ラメトリックオブジェクトを作成するために書いたPythonコードはファイル内に保存されない」と明記されています [1]。この設計は、悪意のあるコードの実行を防ぐための重要なセキュリティ対策です。したがって、ユーザーが期待するような、ファイルを開いたときに自動的に実行されるPythonスクリプトをFCStdファイルに直接「埋め込む」ことはできません [1]。

色データ（プロパティ）の保存とスクリプトロジックの外部管理

色データ自体（例: (1.0, 0.5, 0.0)）は、App.MaterialやカスタムのApp::PropertyColorとしてFCStdファイル内にデータプロパティとして保存されます [1]。しかし、そのデータを視覚的表現に適用する方法をFreeCADに指示するPythonコード（標準の

App.Material解釈でない場合、例えばカスタムプロパティを読み取るロジック）は、ファイル内に保存されません [1]。

FCStdファイルにコードが埋め込めないという事実は、単なる技術的制約に留まらず、FreeCADモデルの「配布」と「互換性」に大きな影響を与えます。特にカスタムプロパティを使用する場合、モデルだけでなく、そのモデルの視覚的解釈に必要なPythonコード（マクロ、カスタムワークベンチ、ViewProviderなど）も別途配布・インストールする必要があるという、運用上の複雑性が発生します。もし、カスタムのApp::PropertyColorを解釈するためにカスタムのViewProviderやマクロを使用する場合、そのViewProviderやマクロはファイルを開くマシン上で別途利用可能でなければなりません [1]。この点は、

App.Materialがより標準的な解釈を持つため、配布の観点から「最も簡単な方法」であることが多いという結論を補強します。

6. 実践的なPythonコード例(統合と解説)

このセクションでは、これまでの説明を具体的なコードで統合し、ユーザーが実際に利用できる実践的な例を提供します。

例**1: GUI**モードでのパーツの**ShapeColor**変更

この例は、ShapeColorビュープロパティの直接的な操作を示しています。これはFreeCADGuiがアクティブである場合にのみ機能します。

Python

```python
import FreeCAD as App
import FreeCADGui as Gui
import Part

# ドキュメントとオブジェクトが存在することを確認
doc = App.newDocument("Color TestDoc")
box = doc.addObject("Part::Box", "MyBox")
box.Shape = Part.makeBox(10, 10, 10)
doc.recompute()

# オブジェクトのGUI表現にアクセス (これはFreeCADがGUIで実行されている場合にのみ機能)
try:
    gui_doc = Gui.activeDocument()
    view_object = gui_doc.getObject("MyBox")
    print(f"Current ShapeColor: {view_object.ShapeColor}")
    view_object.ShapeColor = (1.0, 0.0, 0.0) # 赤に設定
    print(f"New ShapeColor: {view_object.ShapeColor}")
    view_object.ShapeColor = (1.0, 0.0, 0.0, 0.5) # 50%透明の赤
    Gui.updateGui() # 3Dビューを更新して変更を反映
    doc.recompute() # ジオメトリの変更や依存関係がある場合はドキュメントを再計算
    print("Shape color changed to red with 50% transparency.")
except AttributeError:
    FreeCAD.Console.PrintError("GUIがアクティブでないか、オブジェクトが見つかりません。ShapeColorはGUIプロパティです。\n")
```

1

例2: パーツのApp.Materialプロパティ設定(永続性のため)

この例は、Appレベルで色情報を定義する方法を示しています。このデータは永続化され、ロード時にGUIによって解釈されます。これはGUI環境とヘッドレス環境の両方で実行可能です。

Python

```python
import FreeCAD as App
import Part
import os

# AppレベルのFreeCADGuiのインポートやsetupWithoutGUI()は不要
# このスクリプトはFreeCADCmdまたは'FreeCAD -c'で実行されるように設計されています
doc = App.newDocument("Headless Material Doc")
box = doc.addObject("Part::Box", "MyPersistentBox")
box.Shape = Part.makeBox(10, 10, 10)

# App.Materialオブジェクトを作成(これはAppレベルのデータオブジェクト)
material = App.Material()
material.DiffuseColor = (0.0, 1.0, 0.0) # 緑
material.Transparency = 0.3 # 30%透明

# Appオブジェクトにカスタムカラープロパティを追加
# これはデータプロパティであり、永続化され、ヘッドレスでアクセス可能
box.addProperty("App::PropertyColor", "DesiredColor", "Appearance", "表示用の色
").DesiredColor = (0.0, 1.0, 0.0)
box.addProperty("App::PropertyFloat", "DesiredTransparency", "Appearance", "表示用の透明度
").DesiredTransparency = 0.3

doc.recompute()

temp_file_path = os.path.join("/tmp", "headless_material_box.FCStd")
doc.saveAs(temp_file_path)

FreeCAD.Console.PrintMessage(f"ドキュメントがカスタムの色/透明度プロパティと共に保存されました。
パス: {temp_file_path}\n")
# このドキュメントがGUIで開かれた場合、カスタムのViewProviderまたはマクロが
```

```
# 'DesiredColor'と'DesiredTransparency'を読み取り、実際の視覚的表現
# (ShapeColor, Transparency)に適用できる。
```

[1]

例3: 外観プロパティを持つドキュメントの保存

この例は、変更を行った後にドキュメントを保存する方法を示しています。App.Materialやカスタム App::PropertyColorに影響を与えるようなデータプロパティは保存されるのに対し、View プロパティ（永続的なAppバックアップなしで直接設定されたShapeColorなど）は保存されないか、GUIがアクティブな場合にのみ関連するという点が強調されます [1]。FreeCADの標準ファイル形式はFCStdです [1]。

Python

```python
import FreeCAD as App
import Part
import os

# 新しいドキュメントとオブジェクトを作成
doc = App.newDocument("SavedColorDoc")
box = doc.addObject("Part::Box", "MySavedBox")
box.Shape = Part.makeBox(10, 10, 10)

# 方法1: カスタムのApp.PropertyColorを使用(データプロパティ)
# この色は保存され、後でカスタムのViewProviderまたはマクロによって読み取られる可能性がある。
box.addProperty("App::PropertyColor", "StoredColor", "Appearance", "後で表示するための色
").StoredColor = (0.0, 0.5, 1.0) # シアン

# 方法2: カスタムのViewProviderがApp.Materialを読み取るように設定されている場合
# (これはより複雑で、カスタムのPython FeatureまたはViewProviderが必要)
# 標準のPart::Featureの場合、そのViewObject.MaterialプロパティはApp.Materialを指す。
# GUIでこれを設定すると永続化される。ヘッドレスの場合、App.Materialが
# ViewProviderが見つけられるようにAppオブジェクトにアタッチされていることを確認する必要がある。
# 標準オブジェクトの簡単な方法は、GUIでオブジェクトのViewObjectにApp.Materialを直接設定するか、
# App.Materialオブジェクト自体がドキュメントの一部であることを確認することである
```

```
# (例: 名前付きオブジェクトとして。ただし、これは単純な色の場合にはあまり一般的ではない)。
# 標準のPart::Featureの場合、'View'プロパティ (Materialを含む)は、GUIセッションで設定されていれば保存される。
# ヘッドレスの場合、App.Materialオブジェクト自体が暗黙的にリンクされていない場合は管理する必要がある。
# 最も堅牢なヘッドレスな方法は、AppオブジェクトにPropertyColorを直接追加することである。

doc.recompute()
file_path = os.path.join("/tmp", "colored_part.FCStd") # ファイルを保存するパスを定義
doc.saveAs(file_path)

FreeCAD.Console.PrintMessage(f"ドキュメントが保存されました。パス: {file_path}。格納された色プロパティが含まれます。\n")
# 確認するには、FreeCAD GUIで'colored_part.FCStd' を開き、MySaved Boxのプロパティを確認する。
```

[1]

これらのコード例を提示することで、ユーザーは異なるシナリオ（GUIでの即時変更、CUIでのデータ永続化）に対応する具体的な実装方法を理解できます。特に、CUIでの色変更が「データプロパティの保存」に帰結するという核心を、コードを通じて明確に示すことで、概念的な理解を実践的なスキルへと橋渡しします。

7. まとめと推奨事項

FreeCAD Pythonコマンドを使用してパーツの色を変更することは可能です。しかし、その方法と有効性は、FreeCADがグラフィカルユーザーインターフェース (GUI)で実行されているか、ヘッドレス (CUI) 環境で実行されているかによって大きく異なります [1]。

ShapeColorのような視覚的プロパティの直接的な操作は、GUIがアクティブな場合にのみ可能です。ヘッドレスモードでは、これらのGuiモジュールプロパティは利用できません [1]。

FreeCADパーツの色をスクリプト化するためのベストプラクティスは以下の通りです。

インタラクティブな**GUI**セッションの場合**:**

- 即時の視覚的フィードバックを得るには、Gui.activeDocument().getObject("ObjectName").ShapeColor = (R,G,B,A)を使用します [1]。

ヘッドレス **(CUI)** 自動化と永続性の場合**:**

- 主要な方法**:** 色情報をデータプロパティとして保存します。最も堅牢な方法は、カスタムのApp::PropertyColorプロパティをAppオブジェクトに追加することです [1]。
- あるいは、App.Materialオブジェクトとそのグラフィカルプロパティ（DiffuseColorなど）を活用します。オブジェクトのView.MaterialプロパティにApp.Materialを割り当てるにはGUIセッションが必要ですが、App.Material自体の定義は永続化されるAppレベルの操作です。その後、カスタムのViewProviderまたはマクロがこのApp.Materialを読み取り、ドキュメントがGUIで開かれたときに適用できます [1]。
- これらのデータプロパティは、FCStdファイルと共に保存され、後でドキュメントが開かれたときにGUIによって解釈されます [1]。

ヘッドレス視覚出力（例**:**画像生成）の場合**:**

- Gui.setupWithoutGUI()に続いてGui.subgraphFromObject(obj)を使用してシーングラフを生成します [1]。このシーングラフは、オブジェクトのAppレベルのデータ（App.Materialプロパティを含む）を反映し、GUIなしでレンダリングを可能にします [1]。

データ永続性とクロス環境互換性に関する考慮事項:
FreeCADドキュメントと共に保存され、異なるFreeCADセッションや環境（GUIとヘッドレス）間でアクセスする必要がある情報については、常にデータプロパティを優先してください 1。ビュープロパティは、主にGUI内での実行時表示とインタラクションのためのものです 1。GUIとヘッドレスの両方で使用するスクリプトを設計する際には、条件付きロジック（Guiアクセス用のtry-exceptブロックなど）または個別の関数を実装して、異なる環境を処理する必要があります [1]。

ユーザーは「CUIコマンド」について具体的に質問しましたが、CUIでは直接的な視覚的変更（ShapeColor）は不可能であることが示されています [1]。しかし、

App.MaterialやカスタムのApp.PropertyColorはCUIで設定できます。これらのAppレベルのプロパティは、色の「データ」定義であり、視覚的な表示命令ではありません。視覚的な表示は、GUIが存在し、このAppデータを解釈する場合にのみ発生します [1]。このことは、CUIコンテキストでの「色変更」に対するユーザーの理解を再定義します。それは、CUI内でリアルタイムで色変更を見ることではなく、後でGUIによってレンダリングされるように、色情報をプログラムによってモデルのデータ構造に埋め込むことに関するものです。

FreeCADファイルは「セキュリティ上の理由から、埋め込みコードを一切含まない。パラメトリックオブジェクトを作成するために書いたPythonコードはファイル内に保存されない」と述べられています [1]。もし、カスタムの

App::PropertyColorを解釈するためにカスタムのViewProviderやマクロを使用する場合、そのViewProviderやマクロはファイルを開くマシン上で別途利用可能でなければなりません [1]。

これは配布に複雑さを加えるため、

App.Materialが永続的な色にとって最も簡単な方法であることが多いという考え方を強化します。なぜなら、その解釈はより標準化されているからです[1]。

将来の展望:
FreeCADの継続的な開発により、Appレベルでレンダリング固有のプロパティをより直接的に定義する方法が導入され、ヘッドレス機能がさらに強化される可能性があります 1。しかし、基本的なApp/Guiの分離は今後も維持される可能性が高いと予想されます 1。

引用文献

1. FreeCAD Python CUI 色変更_ (1).pdf